Numerical Computation of Partial Differential Equations by Hidden-Layer Concatenated Extreme Learning Machine

Summary of Ni and Dong (2022)

1 Introduction

The Extreme Learning Machine (ELM) method has shown promise in solving computational partial differential equations (PDEs) with high accuracy. However, conventional ELM has a significant limitation: it requires the last hidden layer of the neural network to be wide to achieve high accuracy. If the last hidden layer is narrow, conventional ELM produces poor results regardless of the configuration of the rest of the network.

This paper introduces a modified ELM approach called Hidden-Layer Concatenated ELM (HLConcELM) that overcomes this limitation. HLConcELM can achieve high accuracy for solving linear and nonlinear PDEs regardless of whether the last hidden layer is narrow or wide.

2 Algorithm Description

2.1 Hidden-Layer Concatenated Feedforward Neural Networks (HLConcFNN)

The HLConcELM method is based on a modified feedforward neural network architecture called Hidden-Layer Concatenated FNN (HLConcFNN). Unlike conventional FNNs where only the last hidden layer directly affects the output, HLConcFNN exposes all hidden nodes to the output layer through a logical concatenation operation.

The key architectural difference between conventional FNN and HLConcFNN is illustrated in Figure 1 of the paper. In the conventional FNN, information flows strictly from the input layer through hidden layers sequentially, with only the last hidden layer connecting to the output layer. In HLConcFNN, while the same forward flow exists among hidden layers, a logical concatenation layer is introduced between the last hidden layer and the output layer. This concatenation layer combines the outputs of all hidden nodes from all hidden layers, making them all directly connected to the output nodes.

Given a network with architectural vector $M = [M_0, M_1, ..., M_L]$, where $M_0 = d$ (input dimension) and $M_L = 1$ (output dimension), and activation function σ, the output of HLConcFNN is expressed as:

$$u(x) = \sum_{i=1}^{L-1} \sum_{j=1}^{M_i} \omega_{ij} \eta_j^{(i)}(x) = \sum_{i=1}^{L-1} \boldsymbol{\eta}^{(i)}(x) \boldsymbol{\omega}_i^T = \boldsymbol{\eta}(x) \boldsymbol{\omega}^T \tag{1}$$

where $\eta_j^{(i)}(x)$ is the output field of the j-th node in the i-th hidden layer, and ω_{ij} is the weight coefficient connecting this hidden node to the output node. The logical concatenation is represented by $\boldsymbol{\eta} = [\boldsymbol{\eta}^{(1)}, \boldsymbol{\eta}^{(2)}, ..., \boldsymbol{\eta}^{(L-1)}]$.

2.2 HLConcELM Algorithm

The HLConcELM algorithm combines the HLConcFNN architecture with the core ELM approach:

1. **Random Weight Assignment**: Set weight/bias coefficients in all hidden layers to random values and keep them fixed. - For the i-th hidden layer, weights and biases are randomly generated on $[-R_i, R_i]$, where R_i is a hyperparameter. - The vector $\mathbf{R} = (R_1, R_2, ..., R_{L-1})$ is called the hidden magnitude vector.

2. **Training Process**: - Only the output-layer coefficients (connecting all hidden nodes to the output) are trainable. - These coefficients are determined by solving a linear or nonlinear least squares problem.

3. **PDE Solution Methodology**: - For a PDE system $Lu + F(u) = f(x)$ with boundary conditions $Bu + G(u) = g(x)$ on $\partial\Omega$: - Represent the solution $u(x)$ using HLConcFNN - Substitute this representation into the PDE system - Enforce the equations on collocation points - Solve the resulting system for the output-layer coefficients using linear least squares (for linear PDEs) or nonlinear least squares (for nonlinear PDEs)

4. **Optimal Hyperparameter Computation**: - The hidden magnitude vector $\mathbf{R}$ significantly affects accuracy - Optimal values are determined using differential evolution algorithm - This is a one-time preprocessing step for a given problem

3 Theoretical Findings

3.1 Representation Capacity Properties

One of the significant theoretical contributions of this paper is the analysis of the representation capacity of HLConcFNNs and HLConcELMs. The representation capacity $U(\Omega, M, \sigma)$ refers to the set of all functions that can be exactly represented by a given network architecture.

Three key theoretical findings are established:

1. **Theorem 2.1**: Given an architectural vector $M_1 = (m_0, m_1, ..., m_L)$ with $m_L = 1$, defining a new vector $M_2 = (m_0, m_1, ..., m_{L-1}, n, m_L)$ where $n \geq 1$, the relation $U(\Omega, M_1, \sigma) \subset U(\Omega, M_2, \sigma)$ holds. - This means appending an extra hidden layer never reduces the representation capacity.

2. **Theorem 2.2**: Given an architectural vector $M_1 = (m_0, m_1, ..., m_L)$ with $m_L = 1$, defining a new vector $M_2 = (m_0, m_1, ..., m_{s-1}, m_s+1, m_{s+1}, ..., m_L)$, the relation $U(\Omega, M_1, \sigma) \subset U(\Omega, M_2, \sigma)$ holds. - This means adding nodes to existing hidden layers never reduces the representation capacity.

3. **Theorem 2.3**: For HLConcELMs with random weights, a similar property holds when appending hidden layers, provided the random coefficients are set appropriately.

These properties ensure that HLConcFNNs and HLConcELMs exhibit a hierarchical structure where representation capacity increases (or at least doesn't decrease) as network complexity grows.

3.2 Contrast with Conventional FNNs

The paper notes that conventional FNNs lack the non-decreasing representation capacity property when additional hidden layers are appended. This is a fundamental advantage of the HLConcFNN architecture, as it guarantees that adding complexity to the network (either through wider or deeper architectures) can only enhance its representation ability.

4 Numerical Examples and Results

The paper presents extensive numerical experiments with various linear and nonlinear PDEs to demonstrate the performance of HLConcELM. Five key benchmark problems are presented:

4.1 Variable-Coefficient Poisson Equation

A 2D Poisson equation with variable coefficient:

$$\frac{\partial}{\partial x}\left(a(x,y)\frac{\partial u}{\partial x}\right) + \frac{\partial}{\partial y}\left(a(x,y)\frac{\partial u}{\partial y}\right) = f(x,y) \tag{2}$$

$$\text{with Dirichlet boundary conditions} \tag{3}$$

where $a(x, y) = 2 + \sin(x + y)$

Results showed:

- With network architecture [2,800,50,1] (narrow last hidden layer):
  - Conventional ELM: Maximum error ≈ 85 (no accuracy)
  - HLConcELM: Maximum error $\approx 10^{-8}$ (high accuracy)

- Exponential convergence of HLConcELM errors with increasing collocation points
- Exponential convergence with increasing network width
- HLConcELM effective with both shallow and deep architectures

4.2 Advection Equation

A 1D advection equation plus time:

$$\frac{\partial u}{\partial t} - 2\frac{\partial u}{\partial x} = 0 \tag{4}$$

with periodic boundary conditions and initial condition (5)

Results demonstrated:

- HLConcELM combined with block time marching achieved maximum errors $\approx 10^{-9}$
- Conventional ELM with narrow last hidden layer [2,500,50,1] had maximum errors $\approx 10^{-2}$
- Exponential convergence with respect to both collocation points and network width
- HLConcELM effective on deep networks with multiple narrow hidden layers

4.3 Nonlinear Helmholtz Equation

A 2D nonlinear Helmholtz equation:

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} - 100u + 10\cosh(u) = f(x, y) \tag{6}$$

with Dirichlet boundary conditions (7)

Results showed:

- With network [2,500,30,1] (narrow last hidden layer):
  - Conventional ELM: Maximum error ≈ 10 (poor accuracy)
  - HLConcELM: Maximum error $\approx 10^{-6}$ (high accuracy)
- Similar exponential convergence patterns as observed in linear PDE cases
- HLConcELM demonstrated effective performance even with 5 hidden layers

4.4 Burgers' Equation

The viscous Burgers' equation:

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} = \nu\frac{\partial^2 u}{\partial x^2} \tag{8}$$

$$\text{with appropriate boundary and initial conditions} \tag{9}$$

where $\nu = \frac{1}{100\pi}$

Results demonstrated:

- Local HLConcELM with domain decomposition achieved maximum errors $\approx 10^{-7}$
- For problems with sharp gradients over time, HLConcELM combined with block time marching and domain decomposition maintained high accuracy (maximum errors $\approx 10^{-5}$)
- Conventional local ELM showed only moderate accuracy (maximum errors $\approx 10^{-2}$)

4.5 Korteweg-de Vries (KdV) Equation

The KdV equation:

$$\frac{\partial u}{\partial t} - u\frac{\partial u}{\partial x} + \frac{\partial^3 u}{\partial x^3} = f(x,t) \tag{10}$$

$$\text{with appropriate boundary and initial conditions} \tag{11}$$

Results showed:

- With network [2,800,50,1]:
  - Conventional ELM: Maximum error $\approx 10^2$ (no accuracy)
  - HLConcELM: Maximum error $\approx 10^{-8}$ (high accuracy)
- Maintained exponential convergence with respect to both collocation points and network width
- Effective with deep architectures (3 hidden layers)

5 Implications and Advantages

The paper demonstrates several significant advantages of the HLConcELM method over conventional ELM and other neural network-based PDE solvers:

5.1 Enhanced Representation Capacity

- HLConcELM can effectively utilize all hidden nodes across all hidden layers, not just the last layer
- This provides a richer set of basis functions for representing the solution
- The non-decreasing representation capacity ensures that adding complexity to the network enhances its approximation capability

5.2 Architectural Flexibility

- High accuracy regardless of whether the last hidden layer is narrow or wide
- Effective with both shallow and deep network architectures
- Network design constraints are significantly relaxed compared to conventional ELM

5.3 Computational Performance

- Exponential convergence rates for smooth solutions
- Efficient implementation using forward-mode auto-differentiation
- Training time grows approximately linearly with increasing collocation points
- Capable of handling complex problems through domain decomposition and block time marching

5.4 Superior Accuracy for Complex Problems

- Dramatically outperforms conventional ELM on architectures with narrow last hidden layers
- Maintains high accuracy for problems with sharp gradients
- Effective for both linear and nonlinear PDEs
- Compatible with domain decomposition for complicated geometries

5.5 Systematic Comparison

The paper presents a systematic comparison between HLConcELM and conventional ELM across various architectures and PDE problems:

- For networks with wide last hidden layers: Both methods achieve high accuracy
- For networks with narrow last hidden layers: HLConcELM maintains high accuracy while conventional ELM fails completely
- HLConcELM provides a more robust approach that is less sensitive to architectural choices

This breakthrough addresses a fundamental limitation in conventional ELM methodology, opening up new possibilities for neural network-based PDE solvers with greater flexibility and reliability.